

Final draft

1. Design Requirements

1.1 Problem Statement

- Implement Conv1 layer for simplified LeNet-5
- Input: 8×8 grayscale image (single channel)
- Kernel: 3×3 convolution
- Stride: 2
- Padding: 1
- Output: 4×4 feature map (16 values)

1.2 Hardware Constraints

- 4 MAC units (Multiply-Accumulate)
- Each MAC: 8-bit × 8-bit signed integer multiplication
- 16 bytes on-chip SRAM (dual-port, each port 32-bit = 4 bytes/cycle)
- External memory bandwidth: 1 KB/s (1024 bytes/second)
- Local memory to SRAM transfer: 1 byte per cycle
- No additional hardware (reuse MAC adders for reduction)

1.3 Design Goals

- Maximize throughput (images/second)
- Minimize on-chip memory footprint
- Efficient bandwidth utilization
- Minimize MAC idle time through pipelining

1.4 Data Types

- Input pixels: 8-bit signed integers [-128, 127]
- Weights: 8-bit signed integers [-128, 127]
- Products: 16-bit signed integers
- Accumulator: 20-bit signed integers (to prevent overflow)
- Output: 20-bit signed integers (3 bytes packed)

2. Architecture Assumptions

2.1 Memory assumptions

2.1.1 Memory Assumptions

- External memory is byte-addressable
- Row-major storage for input image
- 8-byte parallel read capability from SRAM
- Input image loaded once, 3×3 windows extracted in hardware
- Weight read for every patch

2.1.2 Dual-Port Access

- Port A: 32-bit read (4 bytes simultaneously)
- Port B: 32-bit write (4 bytes simultaneously)
- Both ports can operate in same cycle
- A dual-port SRAM provides **two independent address ports**, NOT two bytes from one port:

```

```SRAM Layout (16 bytes total):
+-----+
| Bank A (8B) | Bank B (8B) |
+-----+
| Bytes 0-7 | Bytes 8-15 |
+-----+

```

### 2.1.3 Memory Hierarchy

```

External Memory (unlimited)
 |
 | 1 byte/cycle (local memory interface)
 ▼
Local Memory Buffer (staging)
 |
 | 1 byte/cycle to SRAM
 ▼
On-chip SRAM (16 bytes, dual-port)
 |
 | 4 bytes/cycle per port
 ▼
MAC Registers (4 × 2 = 8 bytes)

```

### 2.1.4 Data Loading Strategy

Since SRAM (16 bytes) cannot hold entire image (64 bytes), data is **streamed**:

1. **Image pixels**: Loaded on-demand per patch, not stored entirely
2. **Weights**: 9 bytes loaded per patch (fits in SRAM with pixels)
3. **Working set per patch**: 9 weights + 9 pixels = 18 bytes

- Exceeds 16-byte SRAM → requires **double-buffering**

#### **Double-Buffer Strategy:**

- While MACs compute with Bank A data, load next data into Bank B
- Swap roles each batch

## **2.2 Timing Assumptions**

- Multiplier latency: 2 cycles (pipelined)
- Adder latency: 3 cycles (matching FP32 timing)
- SRAM access: 1 byte of data read/write per cycle
- Register load: Included in operation timing

## **2.3 Arithmetic Assumptions**

- Signed multiplication using Booth encoding or similar
- Two's complement representation
- 20-bit accumulator sufficient for worst-case:
  - Max value of data per product:  $127 \times 127 = 16,129$
  - Max value of 3 products in ACC0:  $3 \times 16,129 = 48,387$
  - Fits in 20 bits (max value of 20 bits = 524,288)
- No saturation needed for integer operations

## **2.4 Control Assumptions**

- Synchronous design, single clock domain
- Sequential processing of patches (no inter-patch pipelining)
- Intra-batch operations can overlap (load, multiply, add)
- Reduction uses MAC adders in ADD-only mode

## **2.5 Hardware Assumptions**

- MAC units can operate in two modes:
  - MAC mode: multiply-accumulate
  - ADD-only mode: bypass multiplier, add 20-bit inputs
- MUX overhead is negligible in critical path
- Multiplier and adder are combinational with stated latencies

# **3. Theory of Operations**

## **COMPLETE PATCH COMPUTATION**

Cycle	Batch	Operation	MAC0	MAC1	MAC2	MAC3	Notes
	-	Loading SRAM					Loading weights
	-	Loading SRAM					Loading patch values
0	0	Load from SRAM	w0,a0	w1,a1	w2,a2	w3,a3	Parallel load
1	0	Multiply (cycle 1/2)	×	×	×	×	Pipeline stage 1
2	0	Multiply (cycle 2/2)	×	×	×	×	Products ready
3	0	Add (cycle 1/3)	+	+	+	+	Start accumulate
4	0	Add (cycle 2/3)	+	+	+	+	Continue
5	0	Add (cycle 3/3)	+	+	+	+	ACC updated (reset)
6	1	Load from SRAM	w4,a4	w5,a5	w6,a6	w7,a7	Next batch
7	1	Multiply (cycle 1/2)	×	×	×	×	
8	1	Multiply (cycle 2/2)	×	×	×	×	
9	1	Add (cycle 1/3)	+	+	+	+	Accumulate mode
10	1	Add (cycle 2/3)	+	+	+	+	
11	1	Add (cycle 3/3)	+	+	+	+	ACC += product
12	2	Load from SRAM	w8,a8	idle	idle	idle	Last operand
13	2	Multiply (cycle 1/2)	×	idle	idle	idle	MAC0 only
14	2	Multiply (cycle 2/2)	×	idle	LOAD	idle	Loading value from ACC3 to ACC2 to overlap
15	2	Add (cycle 1/3)	+	idle	+	idle	
16	2	Add (cycle 2/3)	+	idle	+	idle	

Cycle	Batch	Operation	MAC0	MAC1	MAC2	MAC3	Notes
17	2	Add (cycle 3/3)	+	idle	+	idle	
18	Red	Reduction step 2	idle	+	idle	idle	ACC1 +ACC2
19	Red	(cycle 2/3)	idle	+	idle	idle	
20	Red	(cycle 3/3)	idle	+	idle	idle	
21	Red	Reduction step 3	+	idle	idle	idle	ACC0+ACC1
22	Red	(cycle 2/3)	+	idle	idle	idle	
23	Red	(cycle 3/3)	+	idle	idle	idle	Final sum ready

**Result:** Final sum in ACC1 (or ACC3 depending on design choice).

Can write to output FIFO / memory in cycle 24.

**Key:** × = Multiply in progress · + = Add/accumulate in progress · LOAD = Loading external ACC value for ADD-only mode · idle = MAC not active

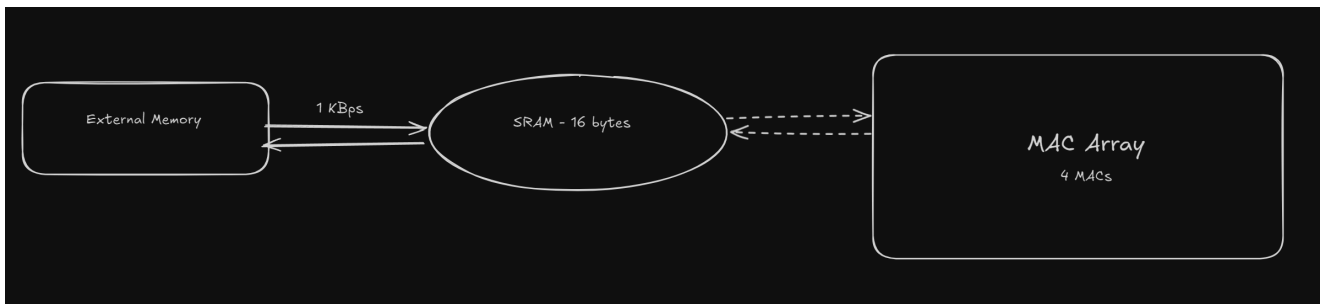
## Summary of operations

Per image computation:

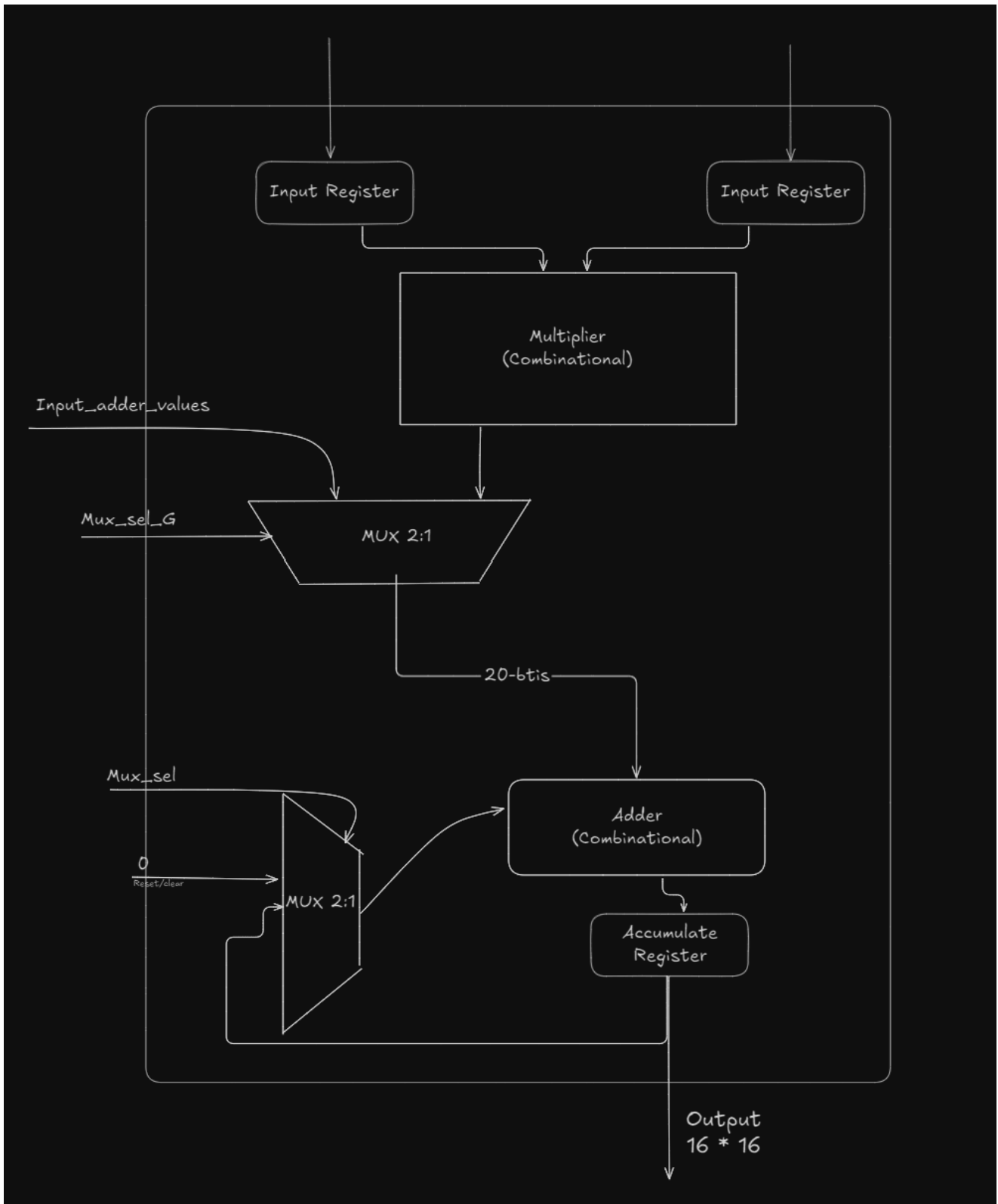
- 16 patches (4×4 output grid)
- Each patch: 24 cycles
- Total:  $16 \times 24 = 384$  cycles per image
- Memory access per image:
  - Input pixels: 64 bytes (8×8, loaded once)
  - Outputs: 16 patches \* 3 bytes = 48 bytes
  - weights called for every patch:  $9 * 16 = 144$
  - Total:  $64 + 144 + 48 = 256$  bytes per image
- Throughput:
  - Bandwidth: 1024 bytes/s
  - Images/s:  $1024 / 256 \approx 4$  images/s
- Clock frequency:
  - Cycles per image: 384
  - Required clock:  $384 \times 4 \approx 1536 = 1.5$  kHz

## 4. Architecture

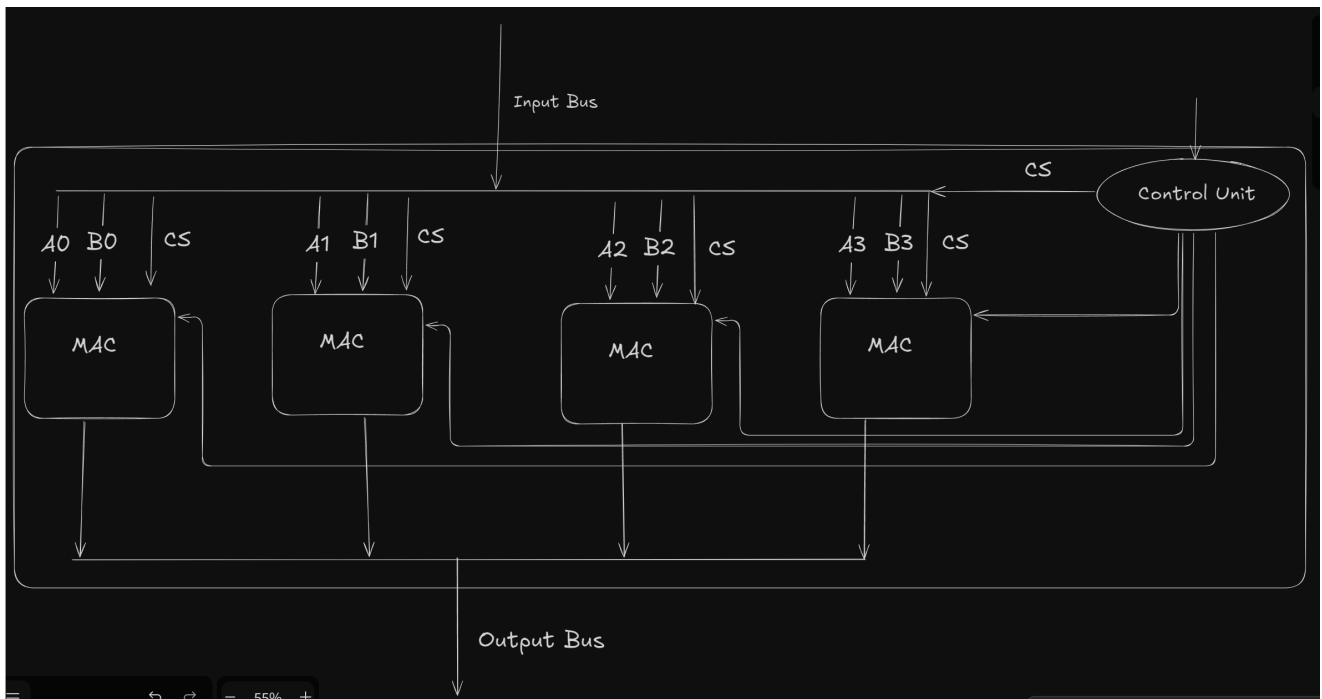
### 4.1 System-Level Block Diagram



## 4.2 MAC Unit Architecture (Your diagram!)



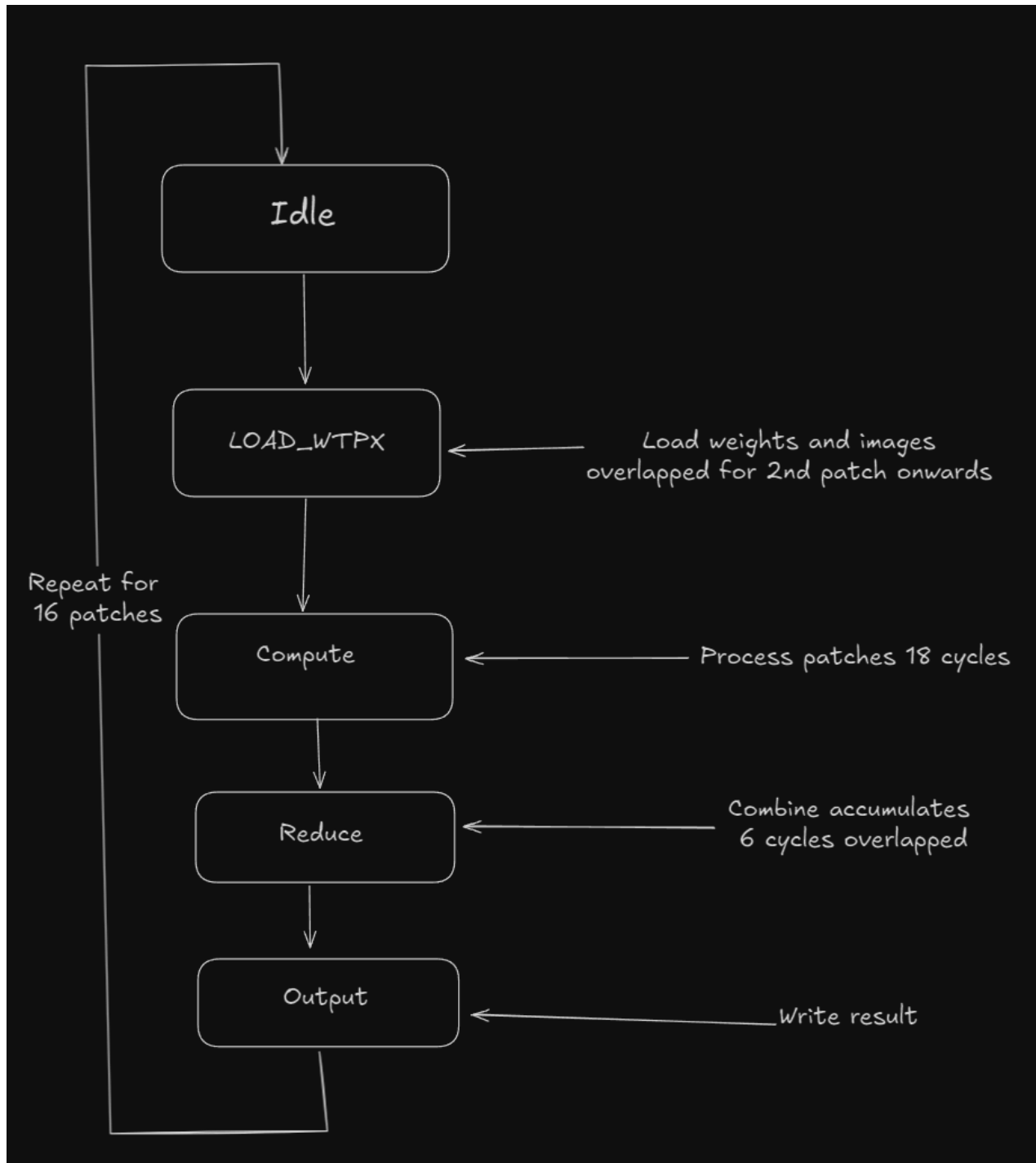
## 4.3 MAC Array (4 units)



## 4.4 Control Unit

- Generates control signals based on current cycle
- Manages batch sequencing (0, 1, 2)
- Switches MAC modes (MAC vs ADD-only)
- Handles SRAM addressing

- Coordinates external memory access



## 4.5 Data Flow

Explain step-by-step with arrows:

1. External memory → SRAM (weights + pixels)
2. SRAM → MAC input registers
3. Registers → Multiplier → Product
4. Product → Adder (with ACC feedback)
5. Adder → ACC register
6. For reduction: ACC → MUX → Adder (bypass mult)
7. Final ACC → Output



## 4.6 Critical Signals

List main control signals:

- clk: System clock (~3.25 kHz)
- reset: Global reset
- mode[1:0]: MAC operation mode
- mux\_sel\_G[3:0]: MUX select for each MAC
- acc\_reset[3:0]: Clear accumulator
- sram\_addr[3:0]: SRAM address
- sram\_we: SRAM write enable
- ext\_mem\_rd: External memory read
- ext\_mem\_wr: External memory write

## 5. Timing scheme